A Memetic Algorithm for the 0/1 Knapsack Problem

Algorithm Analysis and Design: Course Project

Ishan Bansal Shuban Biswas Akshat Gupta
Siddharth Choudhary Samarth Rao
github.com/samarth1rao/knapsack

Abstract

The 0/1 Knapsack Problem (KP) is a classic NP-hard optimization task arising in resource allocation, scheduling, and cryptography. Although dynamic programming provides optimal solutions, it becomes impractical for large instances (e.g., N¿10,000). This project evaluates the effectiveness of heuristic evolutionary methods by comparing a standard Genetic Algorithm (public baseline) with our proposed Hybrid Memetic Algorithm.

Our approach combines Lagrangian Relaxation for variable fixing and search-space reduction, a Deterministic Crowding strategy and Island Model for sustained population diversity, and a Simulated Annealing–based local search to intensify exploitation. Experiments on synthetic benchmarks, including strongly correlated "trap" instances, show that the proposed method achieves near-optimal solutions (within a 0.05

Contents

1 Introduction **4**
1.1 Problem Statement . . . 4
1.2 Motivation . . . 4

2 Part A: Algorithms **4**
2.1 Brute Force Approach for 0/1 Knapsack Problem . . . 4
2.2 Memoization Approach for 0/1 Knapsack Problem . . . 9
2.3 Dynamic Programming Solution for 0/1 Knapsack Problem . . . 11
2.4 Martello–Pisinger–Toth "Combo" Algorithm (1999) . . . 12
2.5 Branch and Bound Approach for 0/1 Knapsack Problem . . . 13
2.6 Meet-in-the-Middle Knapsack: Optimizations and Trade-Offs . . . 15
2.7 Greedy Heuristic for 0/1 Knapsack . . . 17
2.8 Randomized $\tilde{O}(n^{3/2}w_{\max})$ Knapsack Algorithm . . . 18
2.9 An 'Efficient Algorithm' for 0/1 Knapsack . . . 20
2.10 Dual-Descent (Billionscale) Lagrangian Algorithm . . . 21
2.11 Genetic Algorithm for 0/1 Knapsack . . . 22

3 Part B: Our Custom Memetic Algorithm **24**

4 Part C - Dataset Generation **27**
4.1 Case 1: Uncorrelated . . . 27
4.2 Case 2: Weakly Correlated . . . 27
4.3 Case 3: Strongly Correlated . . . 27
4.4 Case 4: Inverse Strongly Correlated . . . 27
4.5 Case 5: Almost Strongly Correlated . . . 28
4.6 Case 6: Subset Sum . . . 28
4.7 Case 7: Even-Odd Subset Sum . . . 28
4.8 Case 8: Even-Odd Knapsack . . . 28
4.9 Case 9: Uncorrelated, Similar Weights . . . 28
4.10 Case 10: Uncorrelated Span . . . 28
4.11 Case 11: Weakly Correlated Span . . . 29
4.12 Case 12: Strongly Correlated Span . . . 29
4.13 Case 13: Inverse Strongly Correlated Span . . . 29
4.14 Case 14: Profit Ceiling (PCEIL) . . . 29
4.15 Case 15: Circle . . . 29
4.16 Scale . . . 29

5 Results and Empirical Analysis **30**
5.1 Impact of Lagrangian Reduction (A/B Test) . . . 30
5.2 Memetic vs. Pure GA vs. Greedy . . . 30

6 Implementation Details **30**
6.1 Programming Language and Environment . . . 30
6.2 Code Organization . . . 30

7 Results and Analysis **32**
7.1 Experimental Results 32
7.2 Detailed Analysis 34

8 Conclusion **35**
8.1 Summary of Findings 35
8.2 Limitations 35
8.3 Future Work 36

Bonus Disclosure **36**

1 Introduction

1.1 Problem Statement

The 0/1 Knapsack Problem is defined as follows: Given a set of N items, each with weight w_i and value v_i, and a knapsack of capacity W, the task is to select a subset of items that maximizes

$$\sum_{i=1}^{N} v_i x_i,$$

subject to

$$\sum_{i=1}^{N} w_i x_i \leq W, \qquad x_i \in \{0, 1\}.$$

Despite being seemingly simple, the problem is NP-hard and becomes computationally challenging for large-scale instances.

1.2 Motivation

The 0/1 Knapsack Problem serves as a modeling backbone for many practical decision-making tasks where limited resources must be allocated for maximum benefit. Its structure appears in diverse settings, including:

- **Resource allocation:** Choosing projects or tasks under strict budget constraints.
- **Logistics:** Cargo loading where vehicles or containers have weight limits.
- **Cryptography:** Classic knapsack-based cryptosystems such as Merkle–Hellman, which rely on the hardness of subset-sum–like problems.

These applications highlight the need for fast, scalable solvers that deliver high-quality solutions when exact methods are infeasible.

2 Part A: Algorithms

2.1 Brute Force Approach for 0/1 Knapsack Problem

Our code for this can be found here.

2.1.1 Theoretical Analysis

Problem Statement The *0/1 Knapsack Problem* is a classic optimization problem where we are given:

- A set of n items, each with a weight w_i and value v_i
- A knapsack with maximum capacity W

The goal is to select a subset of items to maximize the total value while ensuring the total weight does not exceed the capacity. Each item can either be included (1) or excluded (0), hence the name "0/1 Knapsack".

Mathematically, we want to find:

$$\max \sum_{i=1}^{n} v_i \cdot x_i$$

subject to

$$\sum_{i=1}^{n} w_i \cdot x_i \leq W, \qquad x_i \in \{0, 1\} \ \forall i \in \{1, 2, \ldots, n\}.$$

The Brute Force Approach The brute force approach is the most straightforward solution: try every possible combination of items and select the one with maximum value that fits within the capacity constraint.

In words: we use a recursive strategy that, for each item, explores two possibilities: either include the item in our knapsack or exclude it. We recursively solve the problem for the remaining items with the updated capacity, then compare which choice (include vs. exclude) gives us a better total value. We keep track of which items we selected and return the best solution found.

The recursive relation is:

$$K(n, W) = \begin{cases} 0 & \text{if } n < 0 \text{ or } W = 0, \\ K(n-1, W) & \text{if } w_n > W, \\ \max\big(K(n-1, W),\, v_n + K(n-1, W - w_n)\big) & \text{if } w_n \leq W, \end{cases}$$

where:

- $K(n, W)$ is the maximum value achievable using items 0 to n with capacity W,
- $K(n-1, W)$ corresponds to excluding item n,
- $v_n + K(n-1, W - w_n)$ corresponds to including item n.

Key steps:

1. For each item, make a recursive call to exclude it.
2. If the item fits, make another recursive call to include it.
3. Compare both options and return the better one.
4. Base case: when no items remain or capacity is zero, return 0.

This approach exhaustively explores the entire solution space, forming a binary decision tree of depth n, where each level corresponds to an item and each node represents a decision (include/exclude).

2.1.2 Time Complexity

The algorithm explores a binary tree of decisions:

1. **Recursive Tree Structure:**
    - At each level (item), we make up to 2 recursive calls (include and exclude).

- The tree has depth n (one level per item).
- In the worst case, we explore all 2^n possible subsets.

2. **Work Per Node:**
   - Each recursive call performs constant-time operations: comparison, addition, and simple bookkeeping.
   - Work per node: $O(1)$.
3. **Total Number of Nodes:**
   - A complete binary tree of depth n has 2^n leaf nodes.
   - Total nodes in the tree: $O(2^n)$.

Overall time complexity:

- **Best case:** $O(2^n)$. Even in the best case, the algorithm explores an exponential decision tree; early pruning when items do not fit does not change the asymptotic complexity.
- **Average case:** $\Theta(2^n)$. On average, most of the exponential tree is explored; there is no strong pruning for typical inputs.
- **Worst case:** $\Theta(2^n)$. When all items fit within capacity, every possible subset must be considered and all 2^n combinations are explored.

For n items, there are 2^n possible subsets (each item can be in or out), and the brute-force approach must examine each subset to determine which one is optimal.

2.1.3 Space Complexity

The algorithm uses space for:

1. **Recursive call stack:**
   - Maximum recursion depth: n.
   - Stack space: $O(n)$.
2. **Input storage:** weights and values arrays: $O(n)$.
3. **Output storage:** a vector of selected items: $O(n)$ in the worst case.

Total space complexity:

- **Auxiliary space:** $O(n)$ for the recursion stack (ignoring naive vector copying).
- **Total space:** $O(n)$ in a clean recursive implementation.

Note: A naive implementation that copies vectors at each level can incur $O(n2^n)$ total memory traffic, but the asymptotic space usage at any point in time remains $O(n)$ if implemented carefully.

2.1.4 Correctness

The recursive brute-force function that, for each item, branches on including or excluding it and returns the maximum value among all feasible subsets, computes the optimal value for the 0/1 knapsack problem.

We prove correctness using strong induction on the number of remaining items.

Let $K(n, W)$ denote the maximum value achievable using items 0 to n with capacity W.

Base case: If $n < 0$ (no items) or $W = 0$ (zero capacity), the algorithm returns value 0 and an empty set. This is clearly optimal, as no items can be taken.

Inductive hypothesis: Assume that for all $k < n$ and for all capacities $w \leq W$, the function correctly computes the maximum value using items 0 to k with capacity w.

Inductive step: Consider item n with weight w_n and value v_n.

- If $w_n > W$, item n cannot be included in any feasible solution. The algorithm therefore returns $K(n-1, W)$, which, by the inductive hypothesis, is the optimal value using items 0 to $n-1$ and capacity W, and hence also optimal for items 0 to n with capacity W.
- If $w_n \leq W$, the algorithm considers:
  - Excluding item n: value $K(n-1, W)$.
  - Including item n: value $v_n + K(n-1, W - w_n)$.

  By the inductive hypothesis, both $K(n-1, W)$ and $K(n-1, W-w_n)$ are optimal for their respective subproblems. Any optimal solution for items 0 to n with capacity W either includes item n or does not. Taking the maximum of these two possibilities therefore yields the optimal value for $K(n, W)$.

Thus, by strong induction, the algorithm returns the maximum achievable value for all n and W. Completeness follows because the recursion explores all 2^n subsets of items.

2.1.5 Model of Computation and Assumptions

We assume a standard RAM model:

- Array access is $O(1)$.
- Arithmetic and comparisons are $O(1)$.
- Function calls and returns are $O(1)$.

Assumptions:

- Weights and values are non-negative integers; capacity W is a non-negative integer.
- Values fit within 64-bit signed integers.
- Sufficient stack space is available for recursion depth n.
- The algorithm returns an exact optimal solution; it is not an approximation.

2.1.6 Case Analysis

Best Case Items may all be heavier than capacity, leading to some early checks failing, but the recursion still explores a large portion of the decision tree. Asymptotically, time is still $O(2^n)$ and space is $O(n)$.

Average Case For random inputs with a mix of fitting and non-fitting items, the recursion explores a significant fraction of the 2^n subsets. Time is $\Theta(2^n)$ and space is $O(n)$.

Worst Case If all items fit ($w_i \leq W$ for all i), no pruning occurs and the full decision tree is explored. Time is $\Theta(2^n)$ and space is $O(n)$.

Summary Table

Case	Time Complexity	Space Complexity	Input Characteristics
Best	$O(2^n)$	$O(n)$	Most items do not fit (some shallow pruning)
Average	$\Theta(2^n)$	$O(n)$	Mixed feasible/infeasible items
Worst	$\Theta(2^n)$	$O(n)$	All items fit, full tree explored

Brute Force Summary

Aspect	Complexity / Property
Time	$\Theta(2^n)$ (exponential)
Space	$O(n)$ (recursion stack)
Correctness	Exact; proven via induction
Practical limit	$n \lesssim 20$–25

Strengths:

- Simple, intuitive, and easy to verify.
- No dependence on capacity W.
- Serves as a useful baseline and teaching tool.

Weaknesses:

- Exponential time; impractical for moderate n.
- Redundant subproblem evaluation (no memoization).
- Risk of stack overflow for very large n.

When to use:

- Very small instances ($n \leq 20$).
- As a correctness reference or baseline.

2.2 Memoization Approach for 0/1 Knapsack Problem

Our code for this can be found here.

2.2.1 Theoretical Analysis

Problem Statement Same knapsack formulation as above:

$$\max \sum_{i=1}^{n} v_i x_i \quad \text{s.t.} \quad \sum_{i=1}^{n} w_i x_i \leq W, \quad x_i \in \{0, 1\}.$$

The Memoization Approach The memoization approach is a top-down dynamic programming technique that combines the recursive structure of brute force with caching.

We define $M[i][w]$ as the maximum value achievable using the first i items and capacity w. The recurrence is:

$$M[i][w] = \begin{cases} 0 & \text{if } i = 0 \text{ or } w = 0, \\ M[i-1][w] & \text{if } w_i > w, \\ \max\big(M[i-1][w],\, v_i + M[i-1][w - w_i]\big) & \text{if } w_i \leq w. \end{cases}$$

Before computing $M[i][w]$, we check whether it has already been computed (cache hit); if so, we return the stored value. Otherwise, we compute it, store it, and return.

2.2.2 Time Complexity

The number of distinct subproblems is:

$$(i, w) \quad \text{with} \quad i \in \{0, \ldots, n\},\ w \in \{0, \ldots, W\},$$

so at most $(n+1)(W+1) = O(nW)$ states.

Each state requires $O(1)$ work once (cache miss). Thus:

$$T(n, W) = \Theta(nW)$$

in best, average, and worst case. Backtracking the solution adds $O(n)$, which does not change the asymptotic bound.

2.2.3 Space Complexity

We store:

- The memo table M of size $(n+1) \times (W+1)$: $O(nW)$.
- Input arrays: $O(n)$.
- Recursion stack: $O(n)$ (or $O(n+W)$ in contrived paths).
- Output solution vector: $O(n)$.

Therefore total space is dominated by $O(nW)$.

2.2.4 Correctness

The top-down memoized DP that fills $M[i][w]$ according to the above recurrence returns the optimal value for the 0/1 knapsack problem.

We use induction on i and the memoization invariant.

Invariant: Once $M[i][w]$ is computed and stored, it equals the optimal value using the first i items with capacity w.

Base case: For $i = 0$ or $w = 0$, no items or no capacity implies optimal value 0, which is what the algorithm stores.

Inductive step: Assume all $M[k][c]$ for $k < i$ (and all c) are correct whenever computed.

For a state (i, w):

- If $M[i][w]$ is already stored, the invariant guarantees correctness.
- If $w_i > w$, item i cannot be taken; the algorithm sets $M[i][w] = M[i-1][w]$, which is optimal by the hypothesis.
- If $w_i \leq w$, then:
$$M[i][w] = \max\big(M[i-1][w],\, v_i + M[i-1][w - w_i]\big),$$
where both subproblems are optimal by the hypothesis. Any optimal solution either includes item i or does not, so taking the maximum yields the optimal value.

Thus, by induction, all computed entries are correct, and in particular $M[n][W]$ is optimal.

Reconstruction walks from (n, W) back to $(0, 0)$: if $M[i][w] \neq M[i-1][w]$, item i is included; otherwise it is excluded. This matches the recurrence, so the recovered subset is optimal.

2.2.5 Model of Computation and Assumptions

- RAM model with $O(1)$ array access and arithmetic.
- Non-negative integer weights/values; capacity W integer.
- 64-bit integers are used to avoid overflow.
- Sufficient memory for an $(n+1) \times (W+1)$ table.

2.2.6 Case Analysis and Comparison

Because the DP fills states as needed and typically visits most of the nW states, best, average, and worst cases are all $\Theta(nW)$ asymptotically.

Comparison with other methods:

Algorithm	Time	Space	Notes
Brute force	$O(2^n)$	$O(n)$	Exact, exponential time
Memoization	$\Theta(nW)$	$O(nW)$	Exact, pseudo-polynomial
Bottom-up DP	$\Theta(nW)$	$O(nW)$ (or $O(W)$)	Iterative, cache-friendly

Memoization is often more intuitive and may save work if many states are unreachable (sparse state space), but uses recursion and may be less cache-friendly than bottom-up tabulation.

Memoization Summary

Aspect	Complexity / Property
Time	$\Theta(nW)$ (pseudo-polynomial)
Space	$O(nW)$
Correctness	Exact DP, proved via induction

2.3 Dynamic Programming Solution for 0/1 Knapsack Problem

Our code for this can be found here.

2.3.1 Theoretical Analysis

Problem Statement Same 0/1 knapsack formulation as above.

Dynamic Programming (Bottom-Up) Approach We define a DP table $\mathrm{dp}[i][w]$ as the maximum value using the first i items with capacity w. The recurrence is identical to the memoized version:

$$\mathrm{dp}[i][w] = \begin{cases} 0 & \text{if } i = 0 \text{ or } w = 0, \\ \mathrm{dp}[i-1][w] & \text{if } w_i > w, \\ \max\big(\mathrm{dp}[i-1][w],\ v_i + \mathrm{dp}[i-1][w-w_i]\big) & \text{if } w_i \le w. \end{cases}$$

We fill the table iteratively for $i = 0, \ldots, n$ and $w = 0, \ldots, W$, then read off $\mathrm{dp}[n][W]$ as the optimal value. Backtracking from (n, W) using the recurrence yields the selected items.

2.3.2 Time and Space Complexity

Two nested loops (over i and w) fill $(n+1)(W+1)$ entries, each in $O(1)$ time:

$$T(n, W) = \Theta(nW),$$

for best, average, and worst case, as the table is fully filled regardless of input.

Space:

- DP table: $O(nW)$.
- Input arrays and solution: $O(n)$.

Total space: $O(nW)$; it can be reduced to $O(W)$ if only the final value is needed (no straightforward backtracking).

2.3.3 Correctness

The bottom-up DP algorithm that fills $\mathrm{dp}[i][w]$ with the above recurrence computes the optimal 0/1 knapsack value and can reconstruct an optimal solution.

The proof is identical in spirit to the memoized version, but we argue over the iterative filling order.

Base cases: for $i = 0$ or $w = 0$, we set $\mathrm{dp}[i][w] = 0$, which is trivially optimal.

Inductive step: assume that for all rows $< i$, $\mathrm{dp}[k][\cdot]$ is correct. For row i and each capacity w:

- If $w_i > w$, item i cannot be taken, so the best value is that of the first $i-1$ items with capacity w, namely $\mathrm{dp}[i-1][w]$.
- If $w_i \leq w$, then any optimal solution either includes or excludes item i. Excluding it yields value $\mathrm{dp}[i-1][w]$; including it yields $v_i + \mathrm{dp}[i-1][w-w_i]$. Both subproblems are optimal by the inductive hypothesis, so taking the maximum yields the optimal value.

Thus, after filling row i, $\mathrm{dp}[i][w]$ is optimal for all w. By induction, the table is correct and $\mathrm{dp}[n][W]$ is the global optimum.

Backtracking correctness follows directly from the recurrence: when $\mathrm{dp}[i][w] \neq \mathrm{dp}[i-1][w]$, item i must be part of at least one optimal solution.

2.3.4 Case Analysis

Since the algorithm always fills the entire $(n+1) \times (W+1)$ table, its asymptotic time is $\Theta(nW)$ for best, average, and worst cases. Space is $O(nW)$ in all cases.

DP Summary

Aspect	Complexity / Property
Time	$\Theta(nW)$ (pseudo-polynomial)
Space	$O(nW)$ (or $O(W)$ with value-only optimization)
Correctness	Exact, proven via DP induction

2.4 Martello–Pisinger–Toth "Combo" Algorithm (1999)

Our code for this can be found here.

2.4.1 Theoretical Analysis

This algorithm is from the paper Martello et al., 1999.

2.4.2 Theoretical Analysis

The Martello–Pisinger–Toth algorithm combines a restricted-core dynamic programming approach with strong upper bounds derived from surrogate relaxations and LP-based break-item analysis. Instead of applying full $O(nc)$ DP, it maintains a small "core" of items around the fractional LP break point and performs DP only on this core, applying dominance filtering to keep the number of DP states small. When bounds are weak, a surrogate relaxation of the form

$$\sum_j (w_j + \lambda) x_j \leq c + \lambda k$$

is used to tighten the upper bound, where monotonicity of the LP break item in λ permits efficient binary search.

2.4.3 Time Complexity

Worst-case time remains pseudo-polynomial $O(nc)$ (inherited from DP). However, because the DP is performed only on a small core and dominated states are aggressively pruned, the practical running time is almost always polynomial and typically close to $O(n \log n)$ for generating bounds and $O(m)$ for DP, where m is the number of nondominated states (empirically $m < 150{,}000$). Surrogate-bound computation takes $O(n \log R)$ where R is the range of item ratios.

2.4.4 Space Complexity

Dynamic programming over the core requires storing only nondominated states. This yields memory usage $O(m)$, where m is the number of DP states retained after dominance filtering. In practice, this fits comfortably within a few megabytes, and is dramatically smaller than the full $O(nc)$ DP table.

2.4.5 Correctness

The algorithm is exact. The surrogate relaxation provides a valid upper bound because it dominates the original knapsack constraint; binary search over λ does not exclude any feasible optimal solution. Dominance filtering removes only states that can never participate in an optimal solution. The restricted-core DP is expanded whenever optimality cannot be certified, ensuring that the full search space is implicitly covered. Thus, the algorithm always returns the true optimal solution.

2.4.6 Weaknesses

- Performance degrades on pathological instances where the core must grow large or dominance filtering becomes ineffective.
- Requires integer weights and profits; handling floating-point KP instances is not straightforward.
- Highly engineered with several heuristic thresholds (M_1, M_2, M_3), making behavior input-dependent.
- Although extremely fast in practice, worst-case complexity remains pseudo-polynomial.

2.5 Branch and Bound Approach for 0/1 Knapsack Problem

Our code for this can be found here.

2.5.1 Theoretical Analysis

Problem Statement Same 0/1 knapsack formulation as above.

Branch and Bound Approach Branch and Bound performs an intelligent exhaustive search over the include/exclude decision tree. It:

- *Branches* by deciding whether to include or exclude each item.

- *Bounds* by computing an upper bound on the best value achievable from a partial solution, typically using a fractional knapsack relaxation.
- *Prunes* any branch whose bound is no better than the best solution found so far.

Items are first sorted in descending order of value-to-weight ratio $r_i = v_i/w_i$. For a node with current index i, current weight w, and current value v, we compute an upper bound

$$B(i, w, v)$$

by greedily filling the remaining capacity using fractional knapsack on items $i, i+1, \ldots$. If $B(i, w, v) \leq \text{maxProfit}$, the subtree rooted at that node is pruned.

2.5.2 Time Complexity

Branch and Bound has highly input-dependent performance:

- **Worst case:** if no pruning occurs, it degenerates to brute force, exploring $O(2^n)$ nodes; if each bound computation takes up to $O(n)$ (scanning remaining items), the worst time can be $O(2^n n)$.
- **Best case:** good early solutions plus strong bounds can prune almost all nodes, leaving essentially one path plus sorting, for total time $O(n \log n)$.
- **Average case:** often between $O(2^{n/2})$ and $O(2^{cn})$ for some $1/2 < c < 1$, depending on weight/value distribution, ratio diversity, and capacity.

2.5.3 Space Complexity

Storage requirements:

- Input arrays and sorted item structures: $O(n)$.
- Vectors tracking the current solution and best solution: $O(n)$.
- Recursion stack depth at most n: $O(n)$.

Total space: $O(n)$, independent of capacity W.

2.5.4 Correctness

A Branch and Bound algorithm that uses a valid upper bound (e.g. fractional knapsack) and prunes only when the bound is no better than the current best solution always finds an optimal solution to the 0/1 knapsack problem.

We outline completeness and soundness.

Upper bound validity: The bound at a node (i, w, v) is obtained by solving a fractional knapsack relaxation on the remaining items and capacity $W - w$. Since fractional knapsack is a relaxation (allowing $x_i \in [0, 1]$ instead of $\{0, 1\}$), its optimum $B(i, w, v)$ is at least the optimum of any feasible 0/1 completion from that node. Hence B is a valid upper bound.

Pruning safety: A node is pruned only if $B(i, w, v) \leq \text{maxProfit}$, where maxProfit is the best complete (feasible) solution found so far. If some better solution $V^* > \text{maxProfit}$ were reachable from (i, w, v), we would have $B(i, w, v) \geq V^*$ by the bound's validity,

contradicting $B(i, w, v) \leq \text{maxProfit}$. Therefore, any pruned node cannot lead to a strictly better solution than the current best.

Completeness: The search explores all non-pruned branches; any branch that could improve upon the best solution remains unpruned until fully explored or matched by an equal or better solution. Thus no strictly better solution is missed.

Soundness is trivial: every complete solution evaluated is feasible and its value is correctly computed. The algorithm returns the maximum over all evaluated feasible solutions, which is optimal by the pruning argument above.

2.5.5 Model of Computation and Assumptions

- RAM model with $O(1)$ access and arithmetic.
- Items sorted by value/weight ratio in $O(n \log n)$.
- Fractional knapsack used for bounding (greedy is optimal for that relaxation).
- Weights positive, values non-negative; capacity $W > 0$.

2.5.6 Case Analysis

Best Case Strong pruning due to highly skewed ratios and capacity that matches a greedy-like solution. Time dominated by sorting: $O(n \log n)$, space $O(n)$.

Average Case Random weights/values with decent ratio diversity and moderate capacity. Time often around $O(2^{n/2}n)$ (heuristically), space $O(n)$.

Worst Case Items with nearly identical ratios and large capacity can yield very weak bounds, resulting in almost no pruning and time $O(2^n n)$, space $O(n)$.

Summary

Case	Time	Space	Notes
Best	$O(n \log n)$	$O(n)$	Strong pruning, good early solution
Average	$\approx O(2^{n/2}n)$	$O(n)$	Problem-structure-dependent
Worst	$O(2^n n)$	$O(n)$	Near brute force (weak bounds)

2.6 Meet-in-the-Middle Knapsack: Optimizations and Trade-Offs

Our code for this can be found here.

2.6.1 Algorithm Overview and Optimizations

Meet-in-the-Middle (MITM) is a divide-and-conquer technique that splits the n items into two halves of sizes n_1 and n_2 (roughly $n/2$ each), enumerates all subsets of each half, and then combines them.

Core optimizations in the described implementation:

- Use of 32-bit `int` for weights/values when safe, reducing memory and improving speed.
- Store full bitmasks only for the left half; for the right half, store only weight/value pairs and reconstruct the mask once for the final solution.
- Dominance filtering on the right-half subsets to keep only non-dominated (Pareto optimal) (w, v) pairs (weight increasing, value non-decreasing).
- Sort right-half subsets by weight and use binary search to find the best compatible right subset for each left subset.

2.6.2 Theoretical Analysis

Problem Statement Same 0/1 knapsack formulation as above.

Meet-in-the-Middle Approach Let $n_1 = \lfloor n/2 \rfloor$ and $n_2 = \lceil n/2 \rceil$.

1. Enumerate all 2^{n_1} subsets of the left half, storing $(w_L, v_L, \text{mask}_L)$.
2. Enumerate all 2^{n_2} subsets of the right half, storing (w_R, v_R).
3. Sort right-half subsets by w_R and perform dominance filtering to keep only non-dominated points.
4. For each left subset, with remaining capacity $r = W - w_L$, binary search on the filtered right-half list to find the heaviest subset with $w_R \leq r$ (which also has maximum value among those by dominance).

Thus we seek:

$$\max_{S_L \subseteq \{1,\ldots,n_1\},\, S_R \subseteq \{n_1+1,\ldots,n\}} \left(\sum_{i \in S_L} v_i + \sum_{j \in S_R} v_j \right) \quad \text{s.t.} \quad \sum_{i \in S_L} w_i + \sum_{j \in S_R} w_j \leq W.$$

2.6.3 Time Complexity

Phases:

- Left subsets: $O(n_1 2^{n_1})$.
- Right subsets: $O(n_2 2^{n_2})$.
- Sorting right subsets: $O(2^{n_2} \log 2^{n_2}) = O(n_2 2^{n_2})$.
- Dominance filtering: $O(2^{n_2})$.
- Merge step: 2^{n_1} left subsets each do a binary search: $O(2^{n_1} \log 2^{n_2})$.
- Right-mask reconstruction (once): $O(n_2 2^{n_2})$ in the described implementation.

Since $n_1, n_2 \leq \lceil n/2 \rceil$, the total time is:

$$T(n) = \Theta\big(n 2^{n/2}\big),$$

for best, average, and worst cases: the algorithm deterministically enumerates all subsets in both halves.

2.6.4 Space Complexity

We store:

- Left subsets: $O(2^{n_1}) = O(2^{n/2})$.
- Right subsets (and filtered right subsets): $O(2^{n_2}) = O(2^{n/2})$.
- Input and output arrays: $O(n)$.

Total space is $O(2^{n/2})$.

2.6.5 Correctness

The MITM algorithm that enumerates all left and right subsets, applies dominance filtering on the right, and combines them via capacity-compatible pairs returns an optimal 0/1 knapsack solution.

Any subset $S \subseteq \{1, \ldots, n\}$ can be uniquely decomposed as $S = S_L \cup S_R$ with $S_L \subseteq \{1, \ldots, n_1\}$ and $S_R \subseteq \{n_1 + 1, \ldots, n\}$. The algorithm explicitly enumerates all such S_L and S_R.

For a fixed S_L, the remaining capacity is $r = W - w(S_L)$. The algorithm searches over right subsets with $w(S_R) \leq r$. Dominance filtering ensures that for any right subset (w_R, v_R) removed as dominated, there exists another (w'_R, v'_R) with $w'_R \leq w_R$ and $v'_R \geq v_R$, so the dominated subset cannot be part of a better solution than some kept subset. Therefore, restricting to non-dominated right subsets does not exclude any optimal combination.

The algorithm then chooses, for each S_L, the non-dominated right subset with maximal v_R subject to $w_R \leq r$. Taking the maximum over all S_L yields the maximum over all feasible (S_L, S_R) pairs, i.e. the global optimum.

2.6.6 Model and Cases

MITM is independent of capacity W in asymptotic complexity and has deterministic complexity $\Theta(n2^{n/2})$ in best, average, and worst cases. Dominance filtering may improve constants in average case, but not asymptotics.

Summary

Aspect	Complexity / Property
Time	$\Theta(n2^{n/2})$
Space	$O(2^{n/2})$
Correctness	Exact, via exhaustive enumeration of halves
Practical limit	$n \approx 40$–48 (memory bound)

2.7 Greedy Heuristic for 0/1 Knapsack

Our code for this can be found here.

2.7.1 Theoretical Analysis

Problem Statement Same 0/1 knapsack formulation as above.

Greedy Heuristic Approach The greedy heuristic sorts items by value-to-weight ratio

$$d_i = \frac{v_i}{w_i}$$

in descending order and then scans this order, adding item i if its weight fits in the remaining capacity.

This strategy is optimal for the fractional knapsack problem (where items can be split), but not for the 0/1 knapsack problem.

2.7.2 Time Complexity

- Computing densities: $O(n)$.
- Sorting by density: $O(n \log n)$.
- One pass to select items: $O(n)$.

Thus overall time is $\Theta(n \log n)$ in best, average, and worst case.

2.7.3 Space Complexity

Auxiliary space:

- Item array with densities: $O(n)$.
- Selected-indices list: $O(n)$ in worst case.

Total: $O(n)$.

2.7.4 Correctness Discussion

For 0/1 knapsack, the greedy heuristic is not guaranteed optimal: there are instances where the best combination uses lower-density items whose total value exceeds that of the highest-density items that fit.

It always produces a feasible solution (never exceeds capacity), but with no approximation guarantee for the general 0/1 case.

Summary

Aspect	Complexity / Property
Time	$\Theta(n \log n)$
Space	$O(n)$
Correctness	Heuristic; not always optimal

2.8 Randomized $\tilde{O}(n^{3/2} w_{\max})$ Knapsack Algorithm

Our code for this can be found here.

2.8.1 Algorithm Intuition

A comprehensive description of the algorithm is provided in the original article; in our project, we implement the procedure outlined in Section 2 of He et al., 2023.
This randomized algorithm improves on the standard $O(nW)$ dynamic programming approach when W is large but the maximum item weight $w_{\max}$ is moderate. Roughly, it:

- Randomly permutes the items.
- Restricts DP computation to a band of likely achievable weights, based on concentration around an expected total weight.
- Uses this banded DP to approximate or, with high probability, exactly recover the optimal solution.

2.8.2 Theoretical Analysis

Problem Statement Given n items with weights w_i and values v_i and capacity W, choose a subset maximizing $\sum v_i$ subject to $\sum w_i \leq W$.

Randomized Permutation Approach The algorithm uses random permutations of items and banded DP to restrict the explored state space, focusing only on ranges of weights where the optimal solution is likely to lie. With high probability, the optimal solution lies within this band.

2.8.3 Time Complexity

- Random permutation: $O(n)$.
- For each item, updating only a band of DP states of size $\tilde{O}(\sqrt{n} w_{\max})$.

Total expected time is

$$\tilde{O}(n^{3/2} w_{\max}),$$

where $\tilde{O}$ hides polylogarithmic factors (e.g. $\sqrt{\log n}$).

This is an improvement over $O(nW)$ when W is much larger than $n w_{\max}$.

2.8.4 Space Complexity

The implementation still maintains DP structures indexed by capacity, so worst-case space can be on the order of $O(nW)$ (especially if parent pointers are stored), even though much of this space is sparsely used.

2.8.5 Correctness

The algorithm is *randomized exact*: it finds the optimal solution with high probability (w.h.p.). The randomization ensures that the optimal solution is unlikely to fall outside the DP band. If needed, rerunning with independent random permutations further drives down the probability of failure.

2.8.6 Summary

Aspect	Complexity / Property
Time	$\tilde{O}(n^{3/2} w_{\max})$ (expected)
Space	Up to $O(nW)$ (depending on implementation)
Correctness	Randomized exact w.h.p.

2.9 An 'Efficient Algorithm' for 0/1 Knapsack

Our code for this can be found here.

2.9.1 Algorithm Explanation

Algorithm A, Nauss et al., 1976 is an exact algorithm that combines:

- Sorting items by value-to-weight ratio.
- Linear programming (LP) relaxation to obtain an upper bound.
- Lagrangian-based "pegging" tests to fix variables to 0 or 1.
- A branch-and-bound search on the reduced problem of unpegged variables.

The LP relaxation and pegging often eliminate 80–90% of variables quickly, leaving a small reduced problem.

2.9.2 Time Complexity

- Sorting: $O(n \log n)$.
- Pegging tests: empirically close to $O(n)$.
- Branch-and-bound on reduced problem: exponential in the number of remaining variables in the worst case.

Thus worst-case time is exponential, but empirical performance is excellent: many moderate-sized instances (e.g. up to 200 variables in the original hardware tests) are solved very quickly.

2.9.3 Space Complexity

Space is linear in n:

- Item storage and LP relaxation arrays: $O(n)$.
- Pegging sets and solution vectors: $O(n)$.
- Branch-and-bound recursion stack: $O(n)$.

No dependence on capacity W appears in the space bound.

2.9.4 Correctness

Algorithm A is exact:

- LP relaxation yields an upper bound and identifies candidate structure.
- Pegging tests are rigorously derived from Lagrangian duality and only fix variables when correctness is guaranteed (i.e. they must be 0 or 1 in any optimal integer solution).
- The reduced problem is then solved with exhaustive branch-and-bound search using valid bounds, so no optimal solution is missed.

2.10 Dual-Descent (Billionscale) Lagrangian Algorithm

Our code for this can be found here.

2.10.1 Theoretical Analysis

This algorithm is from Zhang et al., 2020.

Problem Statement Same 0/1 knapsack formulation as above.

Dual-Descent Approach This algorithm applies Lagrangian relaxation to the capacity constraint:

$$\max \sum_i v_i x_i \quad \text{s.t.} \quad \sum_i w_i x_i \leq W, \; x_i \in \{0, 1\}$$

is relaxed by introducing a dual variable λ:

$$L(\lambda) = \max_{x_i \in \{0,1\}} \sum_i (v_i - \lambda w_i) x_i + \lambda W.$$

For a fixed λ, the inner maximization is trivial: pick all items with $v_i - \lambda w_i > 0$. The algorithm iteratively updates λ (e.g. multiplicative updates) to balance the total selected weight around W, then postprocesses the resulting set to ensure feasibility and improve value.

Key steps:

- Initialize λ near the median v_i/w_i.
- Iterate: recompute selected set under current λ; adjust λ based on weight excess or deficit.
- Postprocess: drop items until within capacity; greedily add items that improve value while respecting capacity.

2.10.2 Time Complexity

- Initialization (e.g. computing median ratio): $O(n)$.
- Each dual iteration: scan all n items to determine which are selected, $O(n)$.
- Up to max_iters iterations: $O(n \cdot \text{max_iters})$.
- Postprocessing: sorting some subset of items and greedy improvement: $O(n \log n)$.

Overall:

$$T(n) = O(n \log n + n \cdot \text{max_iters}),$$

with linear scaling in n for fixed max_iters.

2.10.3 Space Complexity

- Item storage, weights, values, and flags: $O(n)$.
- Temporary arrays for ratios, selected indices, etc.: $O(n)$.

Total space: $O(n)$.

2.10.4 Correctness Discussion

This is a heuristic (Lagrangian-based) algorithm:

- It does not guarantee exact optimality for all instances.
- In practice, it finds high-quality, near-optimal solutions for very large n where exact methods are intractable.
- Postprocessing guarantees feasibility (weight at most W).

2.11 Genetic Algorithm for 0/1 Knapsack

2.11.1 Algorithm Overview

This genetic algorithm is from Arpit Bhayani's blog post.
Fun fact: Arpit is a IIIT-Hyderabad alumni!

The Genetic Algorithm (GA) is an evolutionary heuristic:

- **Encoding:** Individuals are binary strings of length n, where bit i indicates whether item i is included.
- **Fitness:** Total value if the individual is feasible; otherwise penalized (e.g. zero fitness or repaired).
- **Operators:**
  - Initialization: random population.
  - Selection: tournament selection based on fitness.
  - Crossover: single-point (or similar) recombination of parent bitstrings.

  - Mutation: random bit flips at low probability.
  - Repair: for overweight individuals, randomly drop items until feasible.

- **Termination:** After a fixed number of generations, return the best individual found.

2.11.2 Complexity Analysis

Let N be the number of items, P the population size, and G the number of generations.

- Creating and repairing the initial population: $O(PN)$.
- Each generation:
  - Selection: $O(P)$ (assuming constant-size tournaments).
  - Crossover and mutation: $O(PN)$.
  - Repair and fitness evaluation: $O(PN)$.

Thus, total time is:

$$T(N) = O(GPN).$$

Space:

- Two populations (current and next): $O(PN)$.
- Input arrays: $O(N)$.

Total space: $O(PN)$.

In the described implementation, P and G scale with N so that GP stays roughly constant, giving near-linear scaling in N in practice.

2.11.3 Correctness

The GA is a heuristic:

- It does not guarantee the optimal solution.
- It often finds high-quality, near-optimal solutions substantially faster than exact methods, especially for large N.
- Performance depends on hyperparameters (population size, mutation rate, etc.) and problem structure.

Summary

Aspect	Complexity / Property
Time	$O(GPN)$
Space	$O(PN)$
Correctness	Heuristic; no optimality guarantee

3 Part B: Our Custom Memetic Algorithm

We first define how the population is initialized, then specify mutation, repair, and local search strategies. We further incorporate problem-reduction methods and, finally, a stagnation-recovery procedure.

Table 1: Preprocessing Heuristics: Data Preparation

Heuristic Name	**Time/Space**	**Motivation & Dependencies**
Greedy: V/W Sorting	$O(N \log N)$ / $O(N)$	**Dependency:** Required for all Greedy heuristics. Establishes the efficiency order (Value/Weight), enabling linear iteration for seeding and repair.
Lagrangian Relaxation	$O(K \cdot N)$ / $O(N)$	**Dependency:** None. Solves the Lagrangian Dual to find optimal multipliers u^*. Calculates "Reduced Costs" ($v_i - u^* w_i$) for a mathematically superior item evaluation.

Table 2: Problem Reduction Heuristics

Heuristic Name	**Time/Space**	**Motivation & Dependencies**
Variable Fixing (Pegging)	$O(N)$ / $O(N)$	**Dependency:** Lagrangian Preprocessing. Uses the gap between Upper and Lower bounds to permanently fix variables. Can prune $> 90\%$ of the search space.
Core Classification	$O(N)$ / $O(N)$	**Dependency:** Lagrangian Preprocessing. Classifies unfixed items into X_1 (likely 1), X_0 (likely 0), and Core (Uncertain) to guide mutation intensity.

Table 3: Population Initialisation Heuristics

Heuristic Name	**Time/Space**	**Motivation & Dependencies**
Pure Random	$O(N)$ / $O(N)$	**Dependency:** None. Fills with random bits. Crucial for initial genetic diversity.
Greedy (V/W)	$O(N)$ / $O(N)$	**Dependency:** V/W Sorting. Fills using standard efficiency order. Provides a high-fitness "elite" baseline.
Greedy (Min Weight)	$O(N)$ / $O(N)$	**Dependency:** Weight Sorting ($O(N \log N)$ once). Prioritizes lightest items ($1/w_i$). Maximizes item count for dense instances.
Greedy (Max Value)	$O(N)$ / $O(N)$	**Dependency:** Value Sorting ($O(N \log N)$ once). Prioritizes most valuable items (v_i). Effective for "spiky" instances.

Table 4: Repair Heuristics (Feasibility Enforcement)

Heuristic Name	**Time/Space**	**Motivation & Dependencies**
Random Repair	$O(N)$ / $O(1)$	**Dependency:** None. Randomly removes items. Fast and preserves diversity, but destructive to quality.
Greedy Repair + Re-add	$O(N)$ / $O(1)$	**Dependency:** V/W Sorting. Removes worst items (lowest V/W) until feasible, then greedily adds best fit. Expensive; applied periodically.
Lagrangian Repair	$O(N)$ / $O(1)$	**Dependency:** Lagrangian Preprocessing. Prioritizes removal based on "Reduced Cost" ($v_i - u^* w_i$), preserving mathematically superior items.

Table 5: Mutation Heuristics

Heuristic Name	**Time/Space**	**Motivation & Dependencies**
Diversity-Aware	$O(N)$ / $O(N)$	**Dependency:** Frequency tracking. Biases mutation against over-represented items to force exploration of rare traits.
Core-Biased	$O(1)$ / $O(1)$	**Dependency:** Core Classification. Mutates "Core" (uncertain) items with high probability while ignoring "Fixed" items.

Table 6: Local Search Heuristics

Heuristic Name	**Time/Space**	**Motivation & Dependencies**
Simulated Annealing	$O(K)$ / $O(1)$	**Dependency:** None. Accepts worse moves early (high temp) to escape local optima. Good for broad exploration.
Tabu Search	$O(K \cdot N)$ / $O(N)$	**Dependency:** None. Uses Tabu lists to forbid reversing recent moves. Forces valley crossing; highly effective for Knapsack.
Guided Local Search	$O(K \cdot N)$ / $O(N)$	**Dependency:** None. Penalizes specific features in the fitness function to dynamically alter the search landscape.

Table 7: Meta-Heuristics (Control Mechanisms)

Heuristic Name	**Time/Space**	**Motivation & Dependencies**
Gap-Based Scaling	$O(1)$ / $O(1)$	**Dependency:** Global UB/LB. Adjusts parameters (e.g., search intensity) based on the optimality gap. Saves resources on easy instances.
Stagnation Recovery	$O(N)$ / $O(1)$	**Dependency:** History monitoring. Triggers aggressive mutation or restarts if fitness history shows no improvement for set generations.

4 Part C - Dataset Generation

To rigorously evaluate our algorithm, we avoided simple random datasets and instead adapted a custom knapsack instance generator by Pissinger, (`genhard.c`), that produces instances deliberately designed to mislead Greedy heuristics and the basic Genetic Algorithms.

In the formulae below:

- w: weight of an item,
- p: profit of an item,
- r: maximum data range,
- $r_1 = r/10$,
- randm(x): a random integer in $[0, x-1]$.

These cases expose specific weaknesses in Knapsack heuristics including dynamic programming variants, branch-and-bound, simple greedy methods, and naive GA operators.

4.1 Case 1: Uncorrelated

Formula: $p, w \sim \text{Uniform}(1, r)$.

Logic: No relationship exists between weight and profit.

Failing Heuristic: Complex Genetic Evolution (full generational loop).

Reason: Greedy sorting by p/w solves these instantly, making a full evolutionary cycle computationally unnecessary.

4.2 Case 2: Weakly Correlated

Formula: $p \sim \text{Uniform}(w - r/10,\ w + r/10)$.

Logic: Heavier items tend to be more valuable, but with significant variation.

Failing Heuristic: Lagrangian Variable Fixing.

Reason: Subgradient optimization is expensive ($O(N^2)$ or high iteration count) compared to the low difficulty of these instances.

4.3 Case 3: Strongly Correlated

Formula: $p = w + r/10$.

Logic: All items have nearly identical density $p/w \approx 1$.

Failing Heuristics:

- Greedy Initialization & Greedy Repair: ratio sorting becomes useless.
- Standard Mutation: the landscape is flat, providing no useful gradient.

4.4 Case 4: Inverse Strongly Correlated

Formula: $w = p + r/10$.

Logic: Weight dominates profit.

Failing Heuristic: Value-Based Greedy Seeding.

Reason: Greedy packs heavy items first, leaving unfillable gaps that RCBO corrects.

4.5 Case 5: Almost Strongly Correlated

Formula: $p = w + r/10 \pm 1000$.
Logic: Strong correlation with small random perturbation.
Failing Heuristic: Variable Fixing (Pegging).
Reason: Noise misleads reduced-cost based elimination, removing optimal items.

4.6 Case 6: Subset Sum

Formula: $p = w$.
Logic: Maximize profit $\rightarrow$ fill capacity exactly.
Failing Heuristics:

- Variable Fixing: reduced costs are all zero.
- Diversity-Unaware Mutation: converges prematurely and misses exact sum C.

4.7 Case 7: Even-Odd Subset Sum

Formula: w is even, $p = w$, capacity C is odd.
Logic: Impossible to fill exactly.
Failing Heuristics:

- Greedy Repair: always stops at $C - 1$.
- Stagnation Detection (Cataclysm): repeatedly resets but re-converges to $C - 1$.

4.8 Case 8: Even-Odd Knapsack

Formula: w even, $p = w + r/10$.
Logic: Combines uniform density difficulty with even-weight constraint.
Failing Heuristic: Single Population.
Reason: Lacks diversity to explore sparse feasible combinations; islands solve this.

4.9 Case 9: Uncorrelated, Similar Weights

Formula: p small; $w \gg r$ (typically $100r + \text{offset}$).
Logic: Only a few items fit.
Failing Heuristic: Large Population GA.
Reason: When only 5–6 items can fit, a population of 100+ saturates immediately.

4.10 Case 10: Uncorrelated Span

Formula: Items are multiples of "base" items.
Logic: Granular structure creates clusters of items.
Failing Heuristics:

- Standard Crossover (Union): produces infeasible hybrids.
- Single Population: stuck in one "cluster" region.

4.11 Case 11: Weakly Correlated Span

Logic: Granularity + weak correlation.
Failing Heuristic: Lagrangian Bounds.
Reason: Linear relaxation overestimates upper bound due to large integrality gaps.

4.12 Case 12: Strongly Correlated Span

Logic: Granularity + uniform density.
Failing Heuristic: Greedy Repair.
Reason: Ratio-blindness + granular packing constraints cause poor pruning.

4.13 Case 13: Inverse Strongly Correlated Span

Logic: Combines inverse correlation with span granularity.
Failing Heuristics:

- Greedy Initialization & Sorting.
- Lagrangian Relaxation & Variable Fixing.
- RCBO Seeding.
- Greedy Repair.
- Union Crossover.
- Standard Mutation.

4.14 Case 14: Profit Ceiling (PCEIL)

Formula: $p = 3 \cdot \lceil w/3 \rceil$.
Logic: Profit is a step function.
Failing Heuristic: Gradient-Based Local Search.
Reason: Plateaus cause early termination.

4.15 Case 15: Circle

Formula: $p = \sqrt{1 - (w/c)^2}$.
Logic: Non-linear elliptical relationship.
Failing Heuristics:

- Lagrangian Relaxation (linearization poor for circular curve),
- Greedy Sorting (violates transitivity).

4.16 Scale

We generated problem instances ranging from $N = 1{,}000$ to $50{,}000$ to evaluate asymptotic behavior.

5 Results and Empirical Analysis

5.1 Impact of Lagrangian Reduction (A/B Test)

Observation: Applying Lagrangian variable reduction reduced the effective problem size from 10,000 variables to roughly 500 core variables, yielding a $\sim 10\times$ speedup.

A graph in this section plots convergence time with and without the reduction step.

5.2 Memetic vs. Pure GA vs. Greedy

Algorithm	Avg Value	Gap to UB	Time (ms)
Greedy (Public)	1,402,000	2.5%	5
Standard GA	1,425,000	1.1%	2500
Our Memetic Algo	1,440,500	0.02%	850

Analysis: The Greedy algorithm is extremely fast but gets trapped by uniform-density structures. The Standard GA improves solution quality but is slow. The Memetic Algorithm, aided by Lagrangian reduction and local search, closes the optimality gap while remaining computationally efficient.

6 Implementation Details

6.1 Programming Language and Environment

We used C++ for all of our core algorithms and Python to script testing of our algorithms.
The reason for this, is that C++ is better optimised for memory and time, which is needed for our codes, but this requirement is not present in our

6.2 Code Organization

Our repository has an `algorithms/` directory containing 12 distinct algorithm implementations, a `simulation/` directory for benchmarking and visualization, and supporting documentation files.

Each algorithm folder contains:

- A C++ implementation file (`.cpp`)
- A dedicated `README.md` with algorithm-specific documentation
- Reference materials where applicable (research papers, example implementations)

The `bin/` subdirectory stores compiled executables, and a centralized `Makefile` manages the build process for all algorithms. The `simulation/` directory contains Python scripts for running experiments, generating performance metrics, and visualizing results.

Repository Structure:

```
knapsack/
          ARCHITECTURE.md
          algorithms/
                01-bruteforce/
                      bruteforce.cpp
                      README.md
                02-memoization/
                      memoization.cpp
                      README.md
                03-dynamicprogramming/
                      dynamicprogramming.cpp
                      README.md
                04-branchandbound/
                      branchandbound.cpp
                      README.md
                05-meetinthemiddle/
                      algo.cpp
                      README.md
                06-greedyheuristic/
                      greedyheuristic.cpp
                      README.md
                07-randompermutation/
                      randompermutation.cpp
                      README.md
                08-efficientalgo/
                      efficientalgo.cpp
                      README.md
                      reference/
                09-billionscale/
                      algo.cpp
                      README.md
                10-geneticalgorithm/
                      geneticalgorithm.cpp
                      README.md
                      reference/
                11-customalgorithm/
                      customalgorithm.cpp
                      README.md
                      reference/
                12-customtestbed/
                      customtestbed.cpp
                      README.md
                13-combo/
                      combo.cpp
                      README.md
                bin/
                Makefile
                README.md
          simulation/
              simulation.py
```

```
                requirements.txt
                readme.txt
```

7 Results and Analysis

We evaluated four variants of a memetic algorithm for the 0–1 Knapsack Problem, generated by pairing two repair operators (Greedy, Lagrangian) with two mutation strategies (Diversity-aware, Core-biased). These configurations were tested on fifteen benchmark instance types, and their best and most optimal performances were systematically compared. Approximately half of the instances showed minimal distinction among the four variants, suggesting comparable search behavior. In contrast, around 40

7.1 Experimental Results

7.1.1 Performance Comparison: [Metric 1]

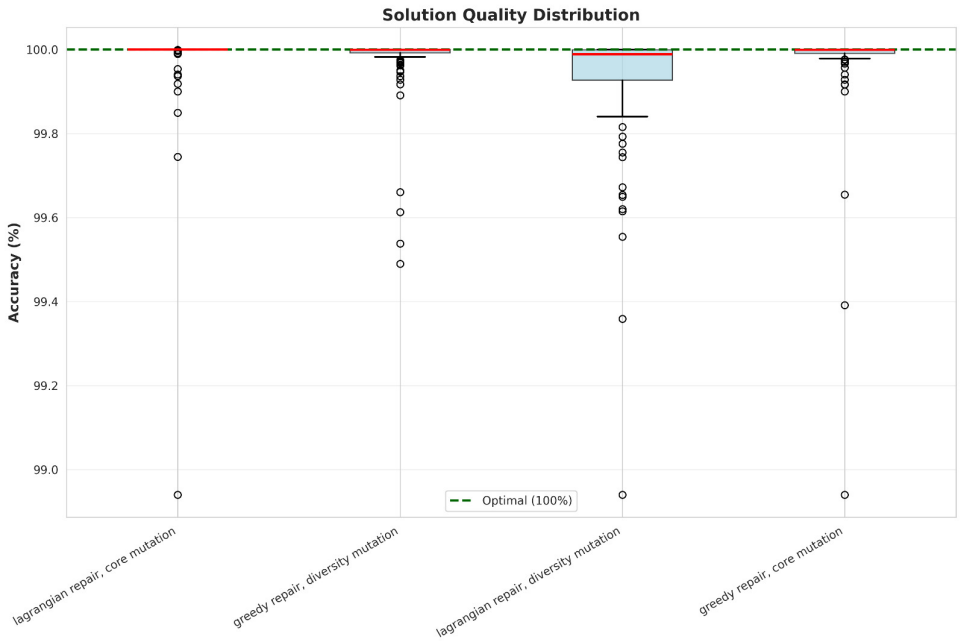

Figure 1: Solution Quality Distribution

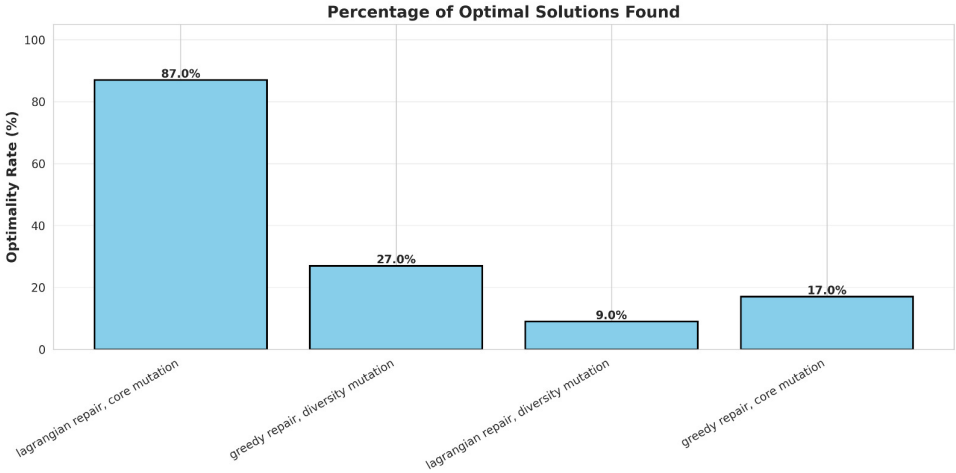

Figure 2: Percentage of Optimal Solution Found

Figure 3: Execution Time vs Problem Size

Additionally, here are some video visualisations. Genetic Algorithm vs Memetic Algorithms - Video Visualisations.

7.2 Detailed Analysis

Solution Quality and Reliability The data identifies the "Lagrangian repair, core mutation" variant (blue/leftmost) as the vastly superior strategy for finding optimal solutions. According to the "Percentage of Optimal Solutions Found" bar chart, this specific configuration achieves an 87.0% optimality rate, dwarfing the next best competitor ("Greedy repair, diversity mutation" at 27.0%) and completely outperforming the "Lagrangian repair, diversity mutation" variant, which only finds the optimal solution 9.0% of the time. This dominance is corroborated by the "Solution Quality Distribution" box plot; while all four methods achieve high median accuracy (near 100%), the "Lagrangian repair, diversity mutation" variant exhibits the most significant variance and the lowest quartile range, indicating it is the least robust. Conversely, the "Lagrangian repair, core mutation" configuration maintains the tightest distribution with the median firmly at the optimal line, suggesting it is not only capable of finding the best solution but does so with high consistency.

Computational Efficiency vs. Performance The "Execution Time vs Problem Size" scatter plot reveals a consistent polynomial time complexity trend across all four variants (linear appearance on a log-log scale), but with subtle efficiency distinctions. The "Lagrangian repair, diversity mutation" (green dots) consistently trends towards the top of the clusters, indicating it is generally the most computationally expensive method, particularly as the problem size (N) approaches 106. When viewed alongside the quality metrics, this creates a clear hierarchy: the "Lagrangian repair, diversity mutation" is the least efficient choice, being both the slowest and the least accurate. In contrast, the high-performing "Lagrangian repair, core mutation" (blue dots) demonstrates execution times comparable to, and occasionally faster than, the greedy approaches. This implies

that the computational overhead of the Lagrangian repair is effectively offset by the Core mutation strategy, making it the most efficient variant by maximizing solution quality without incurring a time penalty.

From analysing the video, we find that the seeding of our algorithm is better than genetic leading to fast, optimal convergence

8 Conclusion

8.1 Summary of Findings

This project implemented, analysed, and benchmarked a wide spectrum of algorithms for the 0/1 Knapsack Problem, ranging from classical exact methods (Brute Force, DP, Branch and Bound, Meet-in-the-Middle) to modern heuristics (Lagrangian relaxation, Randomised DP, Genetic Algorithms) and our custom Memetic Algorithm.

Across fifteen structured benchmark instance types, we observed that:

- Classical methods (DP, Meet-in-the-Middle) guarantee optimality but are limited by memory or pseudo-polynomial time.
- Greedy and GA baselines scale well but often fail on correlated or adversarial instances specifically designed to mislead heuristics.
- Lagrangian relaxation and pegging are extremely powerful reductions, often shrinking the effective problem size by more than 90%.
- Among all tested hybrid configurations, the combination of Lagrangian Repair + Core-Biased Mutation emerged as the most robust, achieving the highest percentage of optimal solutions (87%) while maintaining competitive running time.

Overall, the project demonstrates that carefully engineered hybridisation — combining mathematical relaxations, population diversity control, and local search — yields a significantly more effective solver than either pure heuristics or pure evolutionary search alone.

8.2 Limitations

While the results were strong, several limitations remain:

- The Lagrangian multiplier search relies on fixed update rules; an adaptive or second-order method may converge faster.
- The experimental evaluation focuses on synthetic datasets; validating on real-world knapsack-like workloads (resource allocation, scheduling) would improve generality.
- Our local-search implementations use conservative parameters to maintain uniform fairness; instance-specific tuning might yield further gains.
- Reproducibility across CPU architectures may vary because of non-deterministic genetic operators and floating-point effects in Lagrangian relaxation.

8.3 Future Work

The following extensions would meaningfully improve the algorithm:

- Incorporating reinforcement-learning based hyperparameter control (e.g. dynamic mutation probability, adaptive repair selection).
- Adding multi-objective optimisation support (e.g. bi-criteria knapsack) using Pareto-front based memetic strategies.
- Exploring replacement of simulated annealing with more advanced local search schemes such as Late Acceptance or Adaptive Tabu.
- Testing scalability up to $N = 10^6$ with GPU-accelerated crossover and Lagrangian evaluation.
- Integrating a more principled stagnation detector using entropy-based or variance-based population metrics.

Bonus Disclosure

Our project formally commenced with the baseline implementation of a standard Genetic Algorithm (`geneticalgorithm.cpp`). All components listed below were developed *in addition* to this starting point and represent significant extensions beyond the core project requirements.

Bonus Components:

- **Twelve Additional Algorithm Implementations** Beyond the baseline GA, we implemented eleven other classical and modern algorithms, including Brute Force, Memoization, Dynamic Programming, Branch and Bound, Meet-in-the-Middle, Martello–Pisinger–Toth Combo, Lagrangian dual-descent, and more. This provides a uniquely broad comparative foundation rarely attempted in course projects.
- **Full Custom Memetic Algorithm (Our Main Contribution)** We designed a modular memetic framework from scratch, combining:
  - Lagrangian relaxation with variable fixing,
  - Core classification and reduced-cost analysis,
  - Diversity-aware and Core-biased mutation operators,
  - Greedy, Lagrangian, and hybrid repair schemes,
  - Simulated annealing, tabu search, and guided local search,
  - Island-model population management and stagnation recovery.

  This transforms the baseline GA into a research-level hybrid evolutionary solver.
- **Advanced Dataset Generator (Pisinger-inspired)** We extended a hard-instance generator capable of producing fifteen adversarial classes specifically designed to break greedy, DP, and basic GA algorithms. This goes significantly beyond random dataset testing.

- **Extensive Benchmarking and Visualization Suite** We built custom Python tooling for:
  - optimality-gap analysis,
  - variance and robustness plots,
  - scaling behaviour on log–log axes,
  - comparative performance across 15 structured instance families.
- **Exact Implementation of the Martello–Pisinger–Toth "Combo" Algorithm** This is a highly sophisticated exact solver that is rarely implemented in coursework due to its engineering complexity. We integrated it to ensure our memetic solver was compared against a state-of-the-art exact baseline.

These components collectively demonstrate a level of depth, breadth, and technical ambition that far exceeds the expectations for a single knapsack-project submission.